

OPERATING SYSTEM

What is an Operating System?

- ⇒ A program that acts as an intermediary between a user of a computer and the computer hardware.
- ⇒ An operating system is a collection of system programs that together control the operations of a computer system.

Some examples of operating systems are UNIX, Mach, MS-DOS, MS-Windows, Windows/NT, Chicago, OS/2, MacOS, VMS, MVS, and VM.

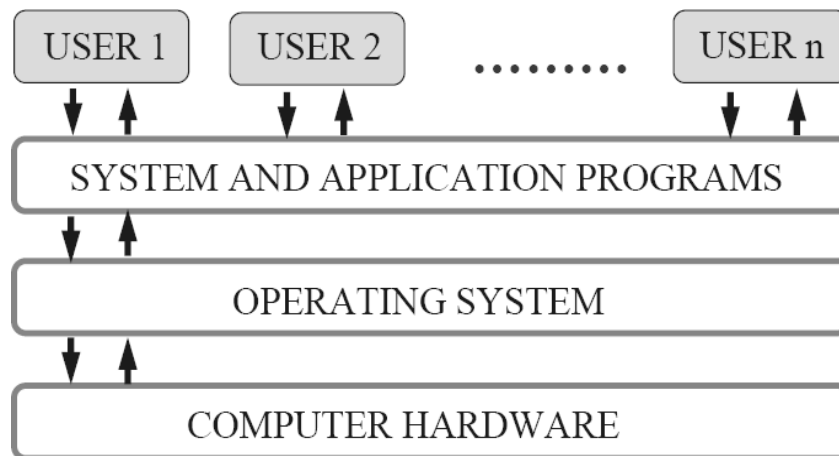
Operating system goals

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use.
- Use the computer hardware in an efficient manner.
-

Computer System Components

1. **Hardware** – provides basic computing resources (CPU, memory, I/O devices).
2. **Operating system** – controls and coordinates the use of the hardware among the various application programs for the various users.
3. **Applications programs** – Define the ways in which the system resources are used to solve the computing problems of the users (compilers, database systems, video games, business programs).
4. **Users** (people, machines, other computers).

Abstract View of System Components



Operating System Definitions

Resource allocator – manages and allocates resources.

Control program – controls the execution of user programs and operations of I/O devices.

Kernel – The one program running at all times (all else being application programs).

Components of OS: OS has two parts. (1) Kernel. (2) Shell.

(1) Kernel is an active part of an OS i.e.; it is the part of OS running at all times. It is a program which can interact with the hardware. Ex: Device driver, dll files, system files etc.

(2) Shell is called as the command interpreter. It is a set of programs used to interact with the application programs. It is responsible for execution of instructions given to OS (called commands).

Operating systems can be explored from two viewpoints: the user and the system.

User View: From the user's point of view, the OS is designed for one user to monopolize its resources, to maximize the work that the user is performing and for ease of use.

System View: From the computer's point of view, an operating system is a control program that manages the execution of user programs to prevent errors and improper use of the computer. It is concerned with the operation and control of I/O devices.

Functions of Operating System

Process Management

A *process* is a program in execution. A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task.

The operating system is responsible for the following activities in connection with process management.

- ◆ Process creation and deletion.
- ◆ process suspension and resumption.
- ◆ Provision of mechanisms for:
 - process synchronization
 - process communication

Main-Memory Management

Memory is a large array of words or bytes, each with its own address. It is a repository of quickly accessible data shared by the CPU and I/O devices.

Main memory is a volatile storage device. It loses its contents in the case of system failure.

The operating system is responsible for the following activities in connections with memory management:

- ◆ Keep track of which parts of memory are currently being used and by whom.
- ◆ Decide which processes to load when memory space becomes available.
- ◆ Allocate and de-allocate memory space as needed.

File Management

A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data.

The operating system is responsible for the following activities in connections with file management:

- ◆ File creation and deletion.
- ◆ Directory creation and deletion.
- ◆ Support of primitives for manipulating files and directories.
- ◆ Mapping files onto secondary storage.
- ◆ File backup on stable (nonvolatile) storage media.

I/O System Management

The I/O system consists of:

- ◆ A buffer-caching system
- ◆ A general device-driver interface
- ◆ Drivers for specific hardware devices

Secondary-Storage Management

Since main memory (*primary storage*) is volatile and too small to accommodate all data and programs permanently, the computer system must provide *secondary storage* to back up main memory.

Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.

The operating system is responsible for the following activities in connection with disk management:

- ◆ Free space management
- ◆ Storage allocation
- ◆ Disk scheduling

Networking (Distributed Systems)

- ◆ A *distributed* system is a collection processor that do not share memory or a clock. Each processor has its own local memory.
- ◆ The processors in the system are connected through a communication network.
- ◆ Communication takes place using a *protocol*.
- ◆ A distributed system provides user access to various system resources.

- ◆ Access to a shared resource allows:
- ◆ Computation speed-up
- ◆ Increased data availability
- ◆ Enhanced reliability

Protection System

- ◆ *Protection* refers to a mechanism for controlling access by programs, processes, or users to both system and user resources.
- ◆ The protection mechanism must:
 - ◆ distinguish between authorized and unauthorized usage.
 - ◆ specify the controls to be imposed.
 - ◆ provide a means of enforcement.

Command-Interpreter System

- Many commands are given to the operating system by control statements which deal with:
 - ◆ process creation and management
 - ◆ I/O handling
 - ◆ secondary-storage management
 - ◆ main-memory management
 - ◆ file-system access
 - ◆ protection
 - ◆ networking
- The program that reads and interprets control statements is called variously:
 - ◆ command-line interpreter
 - ◆ shell (in UNIX)
 - Its function is to get and execute the next command statement.

Operating-System Structures

- System Components
- Operating System Services
- System Calls
- System Programs
- System Structure
- Virtual Machines
- System Design and Implementation
- System Generation

Common System Components

- Process Management
- Main Memory Management
- File Management
- I/O System Management
- Secondary Management
- Networking
- Protection System
- Command-Interpreter System

Evolution of OS

1. Mainframe Systems

Reduce setup time by batching similar jobs Automatic job sequencing – automatically transfers control from one job to another. First rudimentary operating system. Resident monitor

- initial control in monitor
- control transfers to job
- when job completes control transfers back to monitor

2. Batch Processing Operating System:

- This type of OS accepts more than one jobs and these jobs are batched/ grouped together according to their similar requirements. This is done by computer operator. Whenever the computer becomes available, the batched jobs are sent for execution and gradually the output is sent back to the user.
- It allowed only one program at a time.
- This OS is responsible for scheduling the jobs according to priority and the resource required.

3. Multiprogramming Operating System

- This type of OS is used to execute more than one jobs simultaneously by a single processor. it increases CPU utilization by organizing jobs so that the CPU always has one job to execute.
 - The concept of multiprogramming is described as follows:
 - All the jobs that enter the system are stored in the job pool (in disc). The operating system loads a set of jobs from job pool into main memory and begins to execute.
 - During execution, the job may have to wait for some tasks, such as an I/O operation, to complete. In a multiprogramming system, the operating system simply switches to another job and executes. When that job needs to wait, the CPU is switched to *another* job, and so on.
 - When the first job finishes waiting and it gets the CPU back.
 - As long as at least one job needs to execute, the CPU is never idle.
- Multiprogramming operating systems use the mechanism of job scheduling and CPU scheduling.

4. Time-Sharing/multitasking Operating Systems

Time sharing (or multitasking) OS is a logical extension of multiprogramming. It provides extra facilities such as:

- Faster switching between multiple jobs to make processing faster.
- Allows multiple users to share computer system simultaneously.
- The users can interact with each job while it is running.

These systems use a concept of virtual memory for effective utilization of memory space. Hence, in this OS, no jobs are discarded. Each one is executed using virtual memory concept. It uses CPU scheduling, memory management, disc management and security management. Examples: CTSS, MULTICS, CAL, UNIX etc.

5. Multiprocessor Operating Systems

Multiprocessor operating systems are also known as parallel OS or tightly coupled OS. Such operating systems have more than one processor in close communication that sharing the computer bus, the clock and sometimes memory and peripheral devices. It executes multiple jobs at same time and makes the processing faster.

Multiprocessor systems have three main advantages:

- **Increased throughput:** By increasing the number of processors, the system performs more work in less time. The speed-up ratio with N processors is less than N.
- **Economy of scale:** Multiprocessor systems can save more money than multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.
- **Increased reliability:** If one processor fails to done its task, then each of the remaining processors must pick up a share of the work of the failed processor. The failure of one processor will not halt the system, only slow it down. The ability to continue providing service proportional to the level of surviving hardware is called **graceful degradation**. Systems designed for graceful degradation are called **fault tolerant**.

The multiprocessor operating systems are classified into two categories:

- Symmetric multiprocessing system
- Asymmetric multiprocessing system
- ☞ In symmetric multiprocessing system, each processor runs an identical copy of the operating system, and these copies communicate with one another as needed.
- ☞ In asymmetric multiprocessing system, a processor is called master processor that controls other processors called slave processor. Thus, it establishes master-slave relationship. The master processor schedules the jobs and manages the memory for entire system.

6. Distributed Operating Systems

- ☞ In distributed system, the different machines are connected in a network and each machine has its own processor and own local memory.
- ☞ In this system, the operating systems on all the machines work together to manage the collective network resource.
- ☞ It can be classified into two categories:
 - Client-Server systems
 - Peer-to-Peer systems

Advantages of distributed systems.

- ☞ Resources Sharing
- ☞ Computation speeds up – load sharing
- ☞ Reliability
- ☞ Communications
- ☞ Requires networking infrastructure.
- ☞ Local area networks (LAN) or Wide area networks (WAN)

7. Desktop Systems/Personal Computer Systems

- ☞ The PC operating system is designed for maximizing user convenience and responsiveness. This system is neither multi-user nor multitasking.
- ☞ These systems include PCs running Microsoft Windows and the Apple Macintosh. The MS-DOS operating system from Microsoft has been superseded by multiple flavors of Microsoft Windows and IBM has upgraded MS-DOS to the OS/2 multitasking system.
- ☞ The Apple Macintosh operating system has been ported to more advanced hardware, and now includes new features such as virtual memory and multitasking.

8. Real-Time Operating Systems (RTOS)

- ☞ A real-time operating system (RTOS) is a multitasking operating system intended for applications with fixed deadlines (real-time computing). Such applications include some small embedded systems, automobile engine controllers, industrial robots, spacecraft, industrial control, and some large-scale computing systems.
- ☞ The real time operating system can be classified into two categories:
 - 1. hard real time system and 2. soft real time system.
- ☞ A **hard real-time** system guarantees that critical tasks be completed on time. This goal requires that all delays in the system be bounded, from the retrieval of stored data to the time that it takes the operating system to finish any request made of it. Such time constraints dictate the facilities that are available in hard real-time systems.
- ☞ A **soft real-time** system is a less restrictive type of real-time system. Here, a critical real-time task gets priority over other tasks and retains that priority until it completes. Soft real time system can be mixed with other types of systems. Due to less restriction, they are risky to use for industrial control and robotics.

Operating System Services

Following are the five services provided by operating systems to the convenience of the users.

1. Program Execution

The purpose of computer systems is to allow the user to execute programs. So, the operating system provides an environment where the user can conveniently run programs. Running a program involves the allocating and deallocating memory, CPU scheduling in case of multiprocessing.

2. I/O Operations

Each program requires an input and produces output. This involves the use of I/O. So, the operating systems are providing I/O makes it convenient for the users to run programs.

3. File System Manipulation

The output of a program may need to be written into new files or input taken from some files. The operating system provides this service.

4. Communications

The processes need to communicate with each other to exchange information during execution. It may be between processes running on the same computer or running on the different computers. Communications can be occurred in two ways: (i) shared memory or (ii) message passing

5. Error Detection

An error is one part of the system may cause malfunctioning of the complete system. To avoid such a situation operating system constantly monitors the system for detecting the errors. This relieves the user of the worry of errors propagating to various part of the system and causing malfunctioning.

Following are the three services provided by operating systems for ensuring the efficient operation of the system itself.

I. Resource allocation

When multiple users are logged on the system or multiple jobs are running at the same time, resources must be allocated to each of them. Many different types of resources are managed by the operating system.

II. Accounting

The operating systems keep track of which users use how many and which kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.

III. Protection

When several disjointed processes execute concurrently, it should not be possible for one process to interfere with the others, or with the operating system itself. Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders is also important. Such security starts with each user having to authenticate him to the system, usually by means of a password, to be allowed access to the resources.

System Call:

- System calls provide an interface between the process and the operating system.
- System calls allow user-level processes to request some services from the operating system which process itself is not allowed to do.
- For example, for I/O a process involves a system call telling the operating system to read or write particular area and this request is satisfied by the operating system.

The following different types of system calls provided by an operating system:

Process control

- end, abort
- load, execute

File management

- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

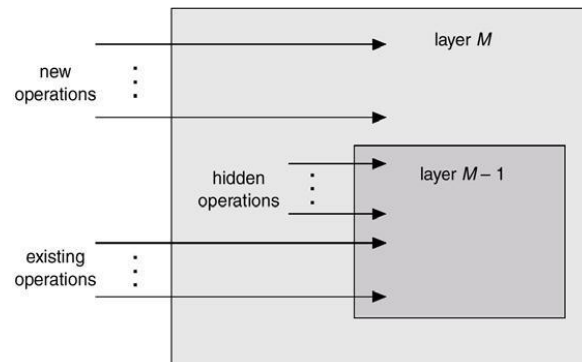
Information maintenance

- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes
- set process, file, or device attributes

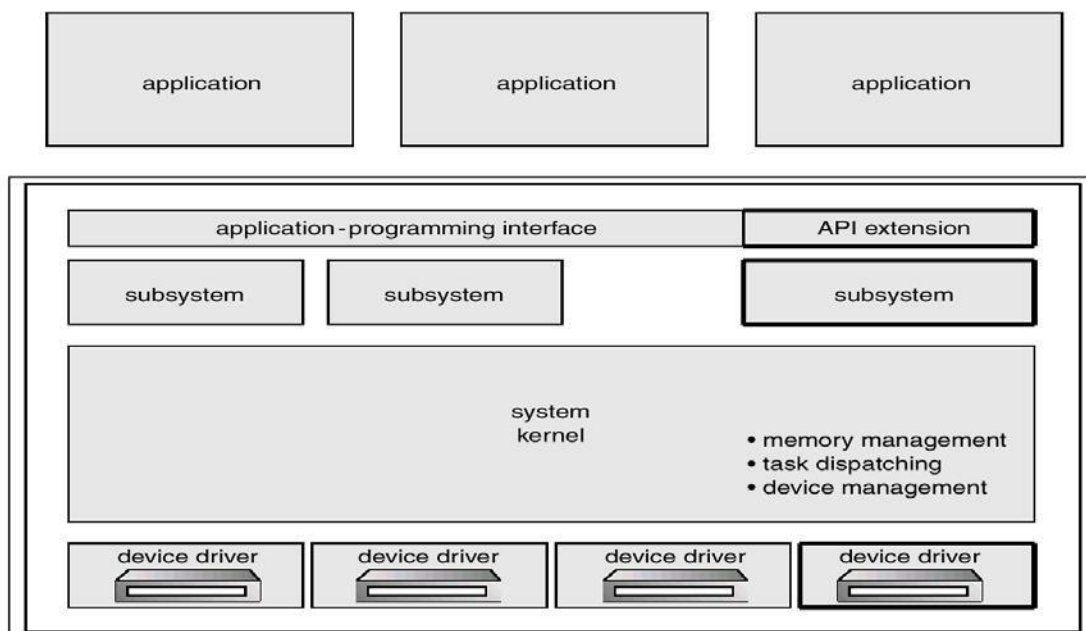
Communications

- create, delete communication connection
- send, receive messages
- transfer status information
- attach or detach remote devices

An Operating System Layer



OS/2 Layer Structure



Microkernel System Structure

Moves as much from the kernel into “user” space.

Communication takes place between user modules using message passing.

Benefits easier to extend a microkernel

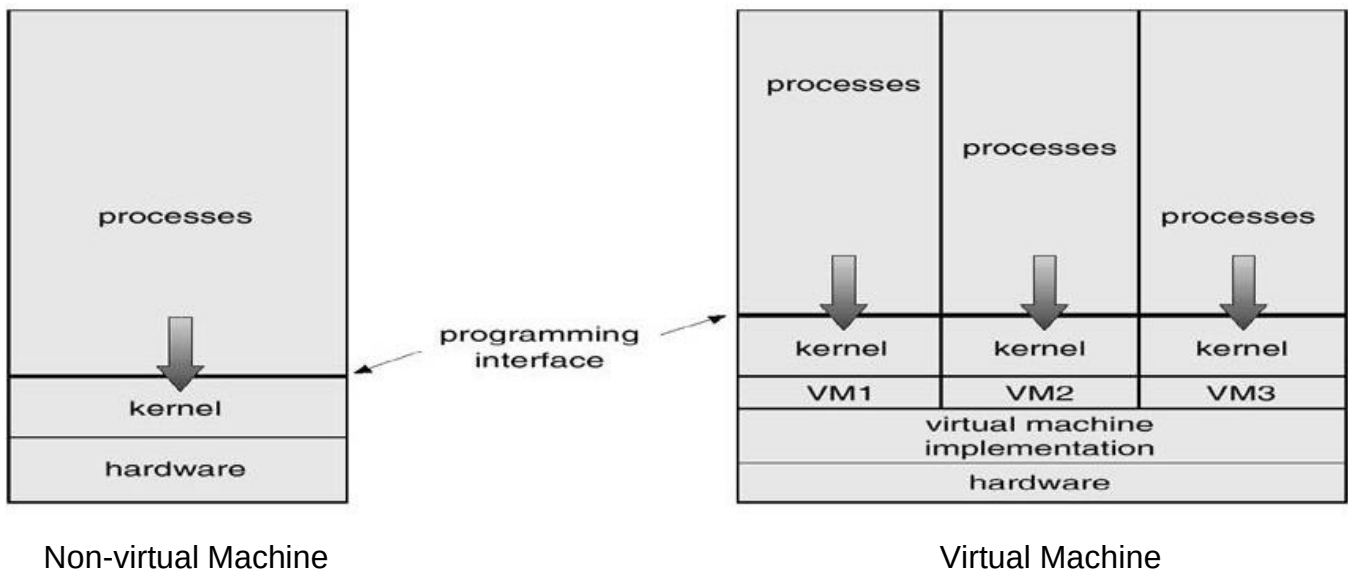
- easier to port the operating system to new architectures
- more reliable (less code is running in kernel mode)
- more secure

Virtual Machines

- A virtual machine takes the layered approach to its logical conclusion. It treats hardware and the operating system kernel as though they were all hardware.
- A virtual machine provides an interface identical to the underlying bare hardware.
- The operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.
- The resources of the physical computer are shared to create the virtual machines.
- CPU scheduling can create the appearance that users have their own processor.
- Spooling and a file system can provide virtual card readers and virtual line printers.
- A normal user time-sharing terminal serves as the virtual machine operator's console.

Advantages/Disadvantages of Virtual Machines

- The virtual-machine concept provides complete protection of system resources since each virtual
- Machine is isolated from all other virtual machines. This isolation, however, permits no direct sharing of resources.
- A virtual-machine system is a perfect vehicle for operating-systems research and development. System development is done on the virtual machine, instead of on a physical machine and so does not disrupt normal system operation.
- The virtual machine concept is difficult to implement due to the effort required to provide an exact Duplicate to the underlying machine.
- System Model



System Generation (SYSGEN)

Operating systems are designed to run on any of a class of machines at a variety of sites with a variety of peripheral configurations. The system must then be configured or generated for each specific computer site; a process sometimes known as system generation (SYSGEN).

SYSGEN program obtains information concerning the specific configuration of the hardware system. To generate a system, we use a special program. The SYSGEN program reads from a given file, or asks the operator of the system for information concerning the specific configuration of the hardware system, or probes the hardware directly to determine what components are there.

The following kinds of information must be determined.

What CPU will be used?

What options (extended instruction sets, floating point arithmetic, and so on) are installed? For multiple-CPU systems, each CPU must be described.

How much memory is available?

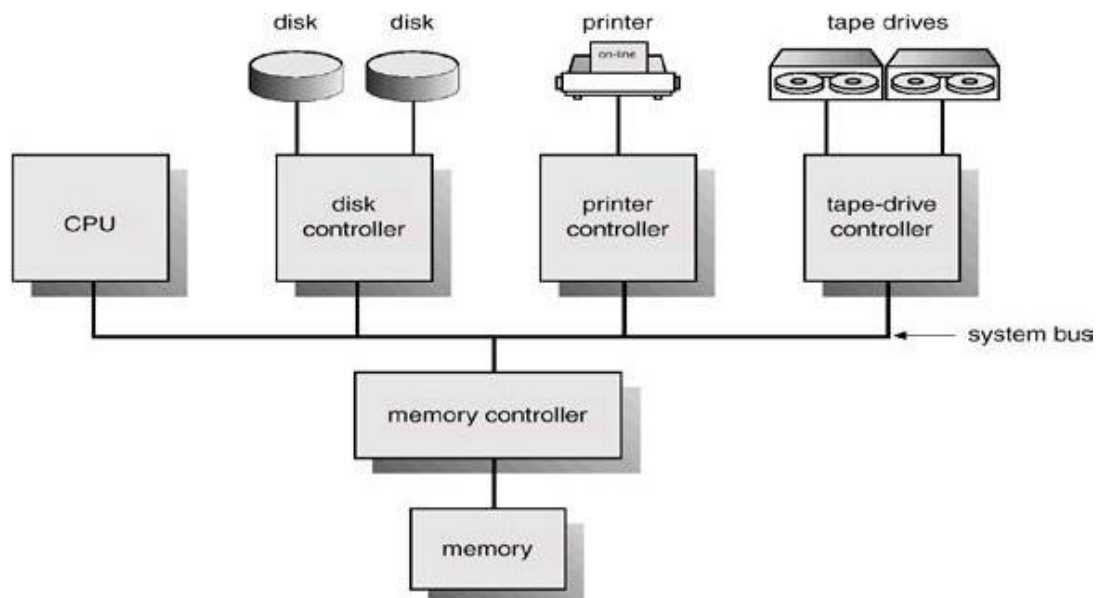
Some systems will determine this value themselves by referencing memory location after memory location until an "illegal address" fault is generated. This procedure defines the final legal address and hence the amount of available memory.

What devices are available?

The system will need to know how to address each device (the device number), the device interrupt number, the device's type and model, and any special device characteristics. What operating-system options are desired, or what parameter values are to be used? These options or values might include how many buffers of which sizes should be used, what type of CPU-scheduling algorithm is desired, what the maximum number of processes to be supported is.

Bootting –The procedure of starting a computer by loading the kernel is known as bootting the system. Most computer systems have a small piece of code, stored in ROM, known as the bootstrap program or bootstrap loader. This code is able to locate the kernel, load it into main memory, and start its execution. Some computer systems, such as PCs, use a two-step process in which a simple bootstrap loader fetches a more complex boot program from disk, which in turn loads the kernel

Computer-System Architecture



Computer-System Operation

- I/O devices and the CPU can execute concurrently.
- Each device controller is in charge of a particular device type.
- Each device controller has a local buffer.
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller.
- Device controller informs CPU that it has finished its operation by causing an *interrupt*.

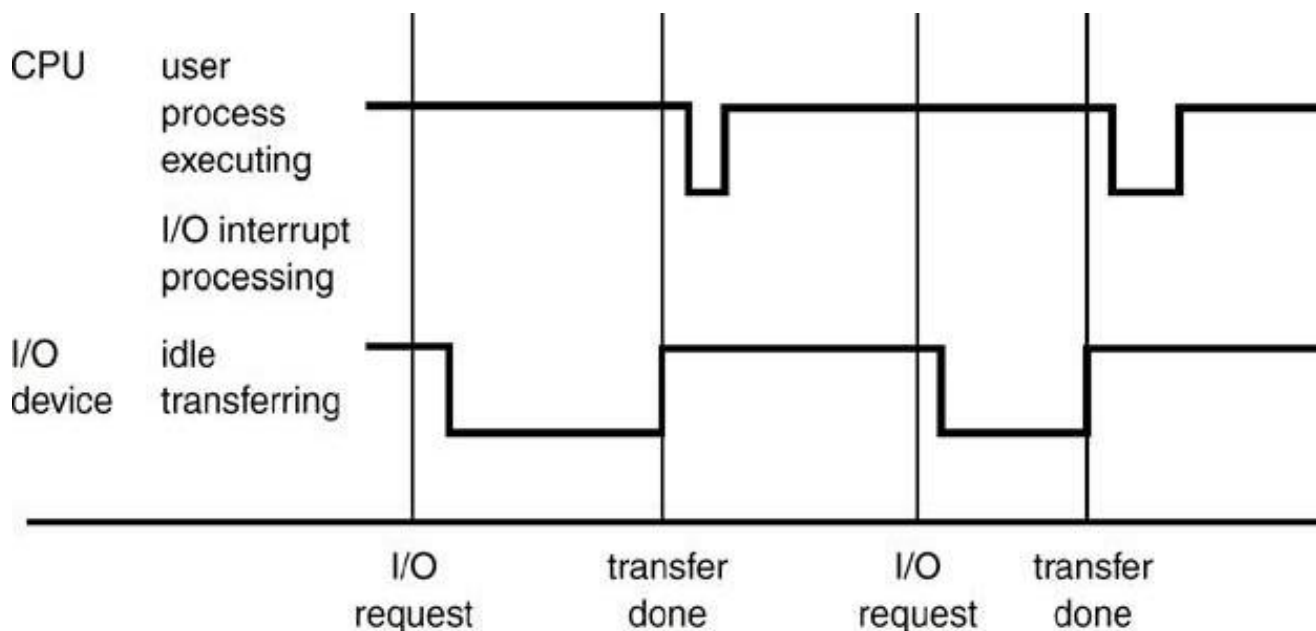
Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the *interrupt vector*, which contains the addresses of all the service routines.
- Interrupt architecture must save the address of the interrupted instruction.
- Incoming interrupts are *disabled* while another interrupt is being processed to prevent a *lost interrupt*.
- A *trap* is a software-generated interrupt caused either by an error or a user request.
- An operating system is *interrupt* driven.
-

Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter.
- Determines which type of interrupt has occurred: *polling*, *vectored* interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt

Interrupt Time Line for a Single Process Doing Output



I/O Structure

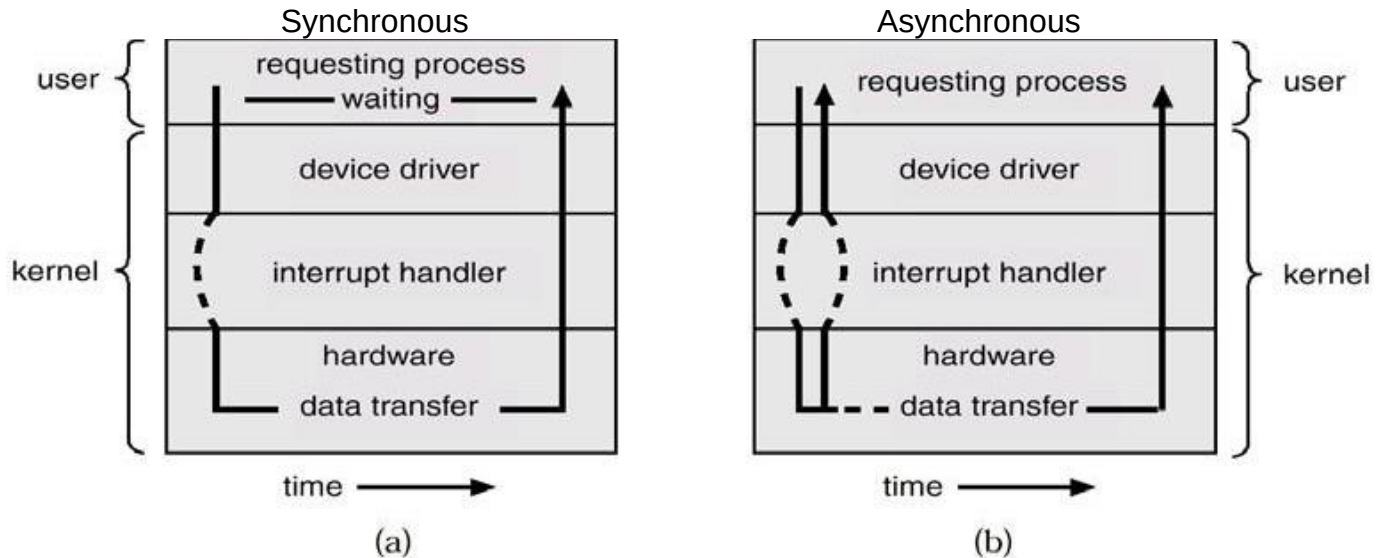
After I/O starts, control returns to user program only upon I/O completion.

- ◆ Wait instruction idles the CPU until the next interrupt
- ◆ Wait loop (contention for memory access).
- ◆ At most one I/O request is outstanding at a time, no simultaneous I/O processing.

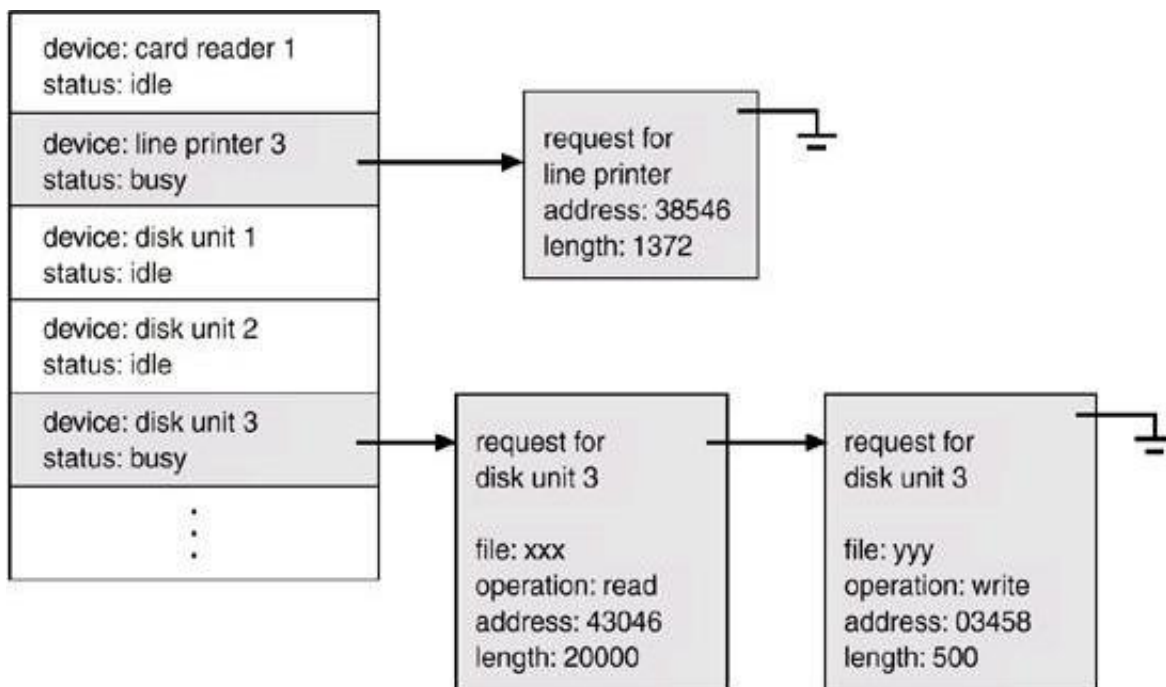
After I/O starts, control returns to user program without waiting for I/O completion.

- ◆ *System call* – request to the operating system to allow user to wait for I/O completion.
- ◆ *Device-status table* contains entry for each I/O device indicating its type, address, and state.
- ◆ Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt.

Two I/O Methods



Device-Status Table



Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds.
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention.
- Only one interrupt is generated per block, rather than the one interrupt per byte.

Storage Structure

Main memory – only large storage media that the CPU can access directly.

Secondary storage – extension of main memory that provides large nonvolatile storage capacity.

Magnetic disks – rigid metal or glass platters covered with magnetic recording material

- ♦ Disk surface is logically divided into *tracks*, which are subdivided into *sectors*.
- ♦ The *disk controller* determines the logical interaction between the device and the computer.

Storage Hierarchy

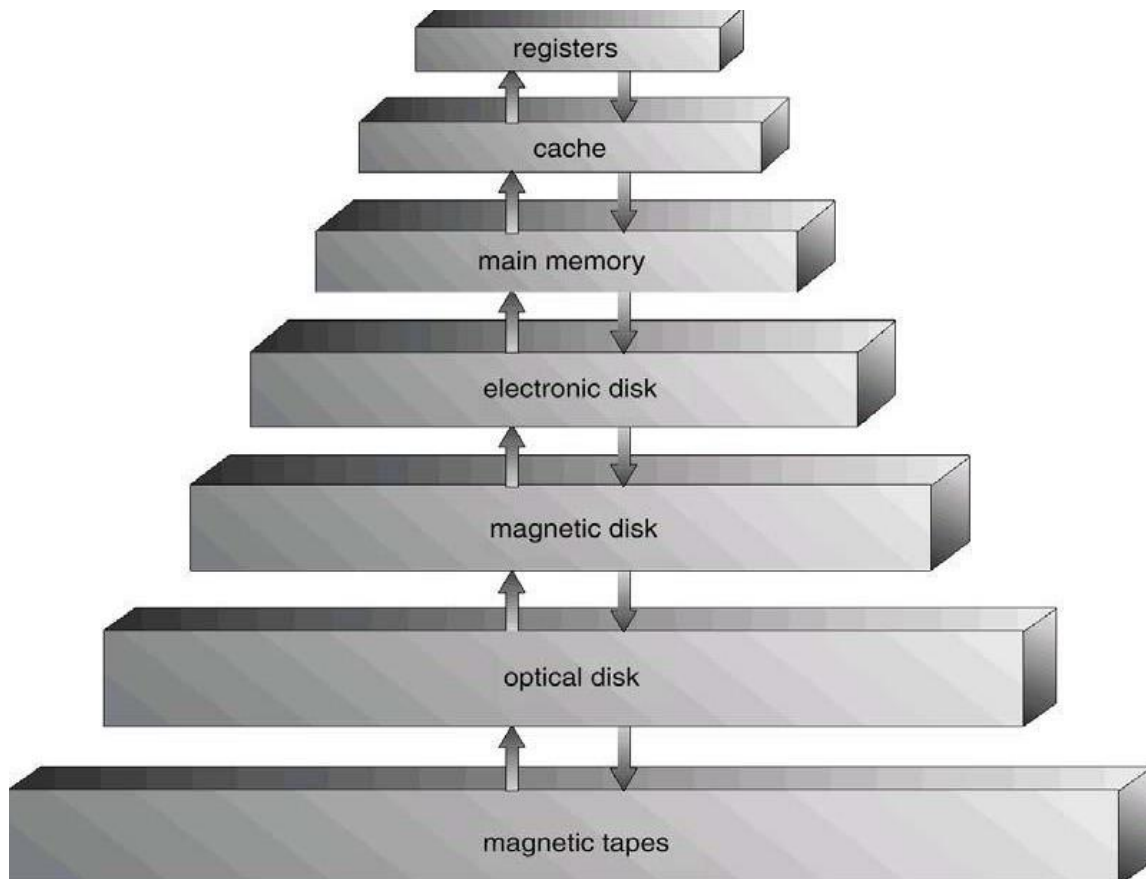
Storage systems organized in hierarchy.

- ♦ Speed
- ♦ Cost
- ♦ Volatility

Caching – copying information into faster storage system; main memory can be viewed as a last *cache* for secondary storage.

- Use of high-speed memory to hold recently-accessed data.
- Requires a *cache management* policy.
- Caching introduces another level in storage hierarchy.
- This requires data that is simultaneously stored in more than one level to be *consistent*.

Storage-Device Hierarchy



Hardware Protection

- Dual-Mode Operation
 - I/O Protection
 - Memory Protection
 - CPU Protection

Dual-Mode Operation

- Sharing system resources requires operating system to ensure that an incorrect program cannot cause other programs to execute incorrectly.
- Provide hardware support to differentiate between at least two modes of operations.
- User mode – execution done on behalf of a user.
- Monitor mode (also kernel mode or system mode) – execution done on behalf of operating system.
- Mode bit added to computer hardware to indicate the current mode: monitor (0) or user (1).
- When an interrupt or fault occurs hardware switches to Privileged instructions can be issued only in monitor mode.
- Monitor user, Interrupt/fault, set user mode I/O Protection
- All I/O instructions are privileged instructions.
- Must ensure that a user program could never gain control of the computer in monitor mode (I.e.a user program that, as part of its execution, stores a new address in the Interrupt vector).

Memory Protection

- Must provide memory protection at least for the interrupt vector and the interrupt service routines.
- In order to have memory protection, add two registers that determine the range of legal addresses a program may access:
 - ◆ Base register – holds the smallest legal physical memory address.
 - ◆ Limit register – contains the size of the range
- Memory outside the defined range is protected.

Hardware Protection

- When executing in monitor mode, the operating system has unrestricted access to both monitor and user's memory.
- The load instructions for the base and limit registers are privileged instructions.

CPU Protection

- Timer – interrupts computer after specified period to ensure operating system maintains control.
 - ◆ Timer is decremented every clock tick.
 - ◆ When timer reaches the value 0, an interrupt occurs.
- Timer commonly used to implement time sharing.
- Time also used to compute the current time.
- Load-timer is a privileged instruction

Process Concept

- Informally, a process is a program in execution. A process is more than the program code, which is sometimes known as the text section. It also includes the current activity, as represented by the value of the program counter and the contents of the processor's registers. In addition, a process generally includes the process stack, which contains temporary data (such as method parameters, return addresses, and local variables), and a data section, which contains global variables.
- An operating system executes a variety of programs:
 - ◆ Batch system – jobs
 - ◆ Time-shared systems – user programs or tasks
- Process – a program in execution; process execution must progress in sequential fashion.
- A process includes: program counter, stack, data section

Process state

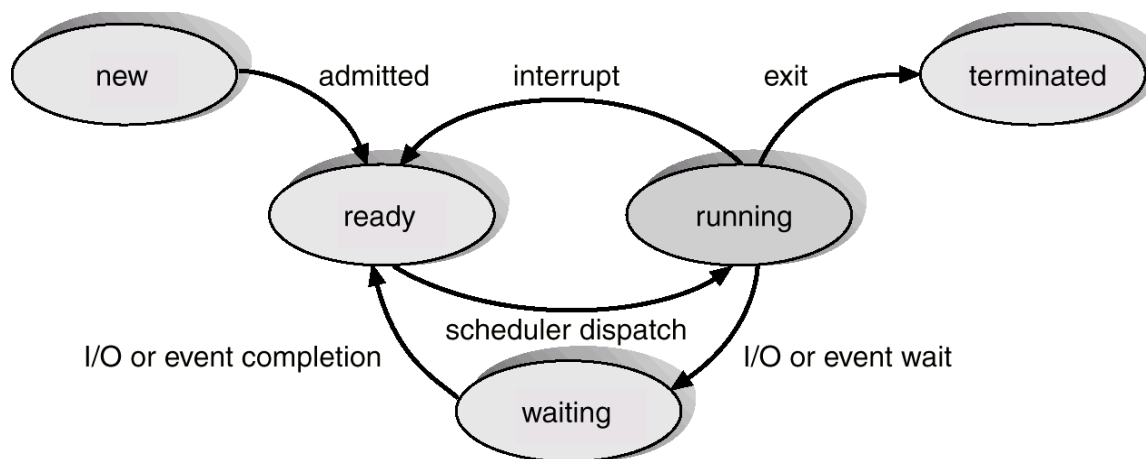
As a process executes, it changes state

- **New State:** The process is being created.
- **Running State:** A process is said to be running if it has the CPU, that is, process actually using the CPU at that particular instant.
- **Blocked (or waiting) State:** A process is said to be blocked if it is waiting for some event to happen such that as an I/O completion before it can proceed. Note that a process is unable to run until some external event happens.
- **Ready State:** A process is said to be ready if it needs a CPU to execute. A ready state process is runnable but temporarily stopped running to let another process run.
- **Terminated state:** The process has finished execution.

What is the difference between process and program?

- 1) Both are same beast with different name or when this beast is sleeping (not executing) it is called program and when it is executing becomes process.
- 2) Program is a static object whereas a process is a dynamic object.
- 3) A program resides in secondary storage whereas a process resides in main memory.
- 4) The span time of a program is unlimited but the span time of a process is limited.
- 5) A process is an 'active' entity whereas a program is a 'passive' entity.
- 6) A program is an algorithm expressed in programming language whereas a process is expressed in assembly language or machine language.

Diagram of Process State



Process Control Block (PCB)

pointer	process state
process number	
program counter	
registers	
memory limits	
list of open files	
⋮	

- Process State
- Program counter
- CPU registers
- CPU scheduling information
- Memory-management information
- Accounting information
- I/O status information

Process state: The state may be new, ready, running, waiting, halted, and SO on.

Program counter: The counter indicates the address of the next instruction to be executed for this process.

CPU registers: The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.

CPU-scheduling information: This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

Memory-management information: This information may include such information as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system.

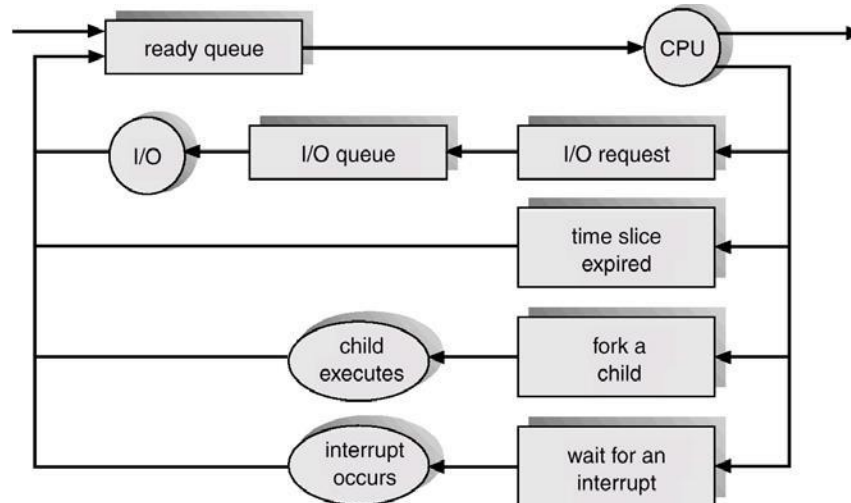
Accounting information: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.

status information: The information includes the list of I/O devices allocated to this process, a list of open files, and so on. The PCB simply serves as the repository for any information that may vary from process to process.

Process Scheduling Queues

- **Job Queue:** This queue consists of all processes in the system; those processes are entered to the system as new processes.
- **Ready Queue:** This queue consists of the processes that are residing in main memory and are ready and waiting to execute by CPU. This queue is generally stored as a linked list. A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue.
- **Device Queue:** This queue consists of the processes that are waiting for a particular I/O device. Each device has its own device queue.

Representation of Process Scheduling



Schedulers

A **scheduler** is a decision maker that selects the processes from one scheduling queue to another or allocates CPU for execution. The Operating System has three types of schedulers:

1. Long-term scheduler or Job scheduler
2. Short-term scheduler or CPU scheduler
3. Medium-term scheduler

Long-term scheduler or Job scheduler

- The long-term scheduler or job scheduler selects processes from discs and loads them into main memory for execution. It executes much less frequently.
- It controls the degree of multiprogramming (i.e., the number of processes in memory).
- Because of the longer interval between executions, the long-term scheduler can afford to take more time to select a process for execution.

Short-term scheduler or CPU scheduler

- The short-term scheduler or CPU scheduler selects a process from among the processes that are ready to execute and allocates the CPU.
- The short-term scheduler must select a new process for the CPU frequently. A process may execute for only a few milliseconds before waiting for an I/O request.

Medium-term scheduler

The medium-term scheduler schedules the processes as intermediate level of scheduling

Processes can be described as either:

- ♦ I/O-bound process – spends more time doing I/O than computations, many short CPU bursts.
- ♦ CPU-bound process – spends more time doing computations; few very long CPU bursts.

Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.
- Context-switch time is overhead; the system does no useful work while switching.
- Time dependent on hardware support.

Process Creation

Parent process create children processes, which, in turn create other processes, forming a tree of processes.

Resource sharing

- ◆ Parent and children share all resources.
- ◆ Children share subset of parent's resources.
- ◆ Parent and child share no resources.

Execution

- ◆ Parent and children execute concurrently.
- ◆ Parent waits until children terminate.

Address space

- ◆ Child duplicate of parent.
- ◆ Child has a program loaded into it.

UNIX examples

- ◆ **fork** system call creates new process
- ◆ **exec** system call used after a **fork** to replace the process' memory space with a new program.

Process Termination

- Process executes last statement and asks the operating system to decide it (**exit**).
- ◆ Output data from child to parent (via **wait**).
- ◆ Process' resources are deallocated by operating system.
 - Parent may terminate execution of children processes (**abort**).
- ◆ Child has exceeded allocated resources.
- ◆ Task assigned to child is no longer required.
- ◆ Parent is exiting.
- ✓ Operating system does not allow child to continue if its parent terminates.
- ✓ Cascading termination.

Cooperating Processes

The concurrent processes executing in the operating system may be either independent processes or cooperating processes. A process is **independent** if it cannot affect or be affected by the other processes executing in the system. Clearly, any process that does not share any data (temporary or persistent) with any other process is independent. On the other hand, a process is **cooperating** if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

Advantages of process cooperation

Information sharing: Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to these types of resources.

Computation speedup: If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Such a speedup can be achieved only if the computer has multiple processing elements (such as CPUS or I/O channels).

Modularity: We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads

Convenience: Even an individual user may have many tasks on which to work at one time. For instance, a user may be editing, printing, and compiling in parallel.

Concurrent execution of cooperating processes requires mechanisms that allow processes to communicate with one another and to synchronize their actions.

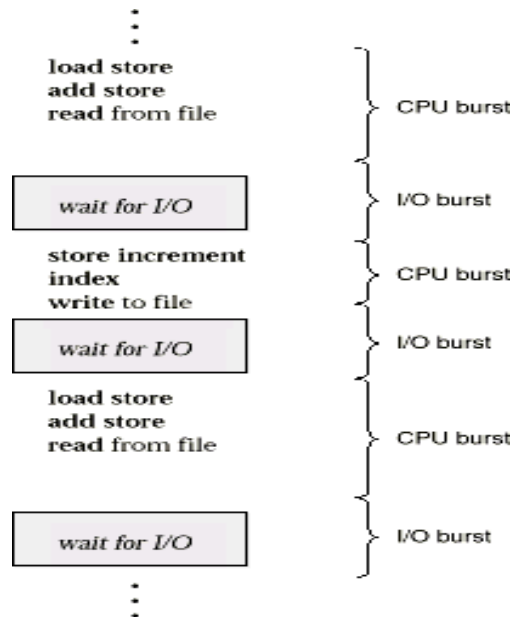
CPU Scheduling

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Multiple-Processor Scheduling
- Real-Time Scheduling
- Algorithm Evaluation

Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU-I/O Burst Cycle – Process execution consists of a *cycle* of CPU execution and I/O wait.
- CPU burst distribution

Alternating Sequence of CPU And I/O Bursts



CPU Scheduler

- Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them.
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state.
 2. Switches from running to ready state.
 3. Switches from waiting to ready.
 4. Terminates.
- Scheduling under 1 and 4 is *non preemptive*.
- All other scheduling is *preemptive*.

Dispatcher

Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:

- switching context
- Switching to user mode
- jumping to the proper location in the user program to restart that program

Dispatch latency – time it takes for the dispatcher to stop one process and start another running.

Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, **not** output (for time-sharing environment)

Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

First-Come, First-Served Scheduling

By far the simplest CPU-scheduling algorithm is the first-come, first-served (FCFS) scheduling algorithm. With this scheme, the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue. When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The running process is then removed from the queue. The code for FCFS scheduling is simple to write and understand. The average waiting time under the FCFS policy, however, is often quite long.

Consider the following set of processes that arrive at time 0, with the length of the CPU-burst time given in milliseconds:

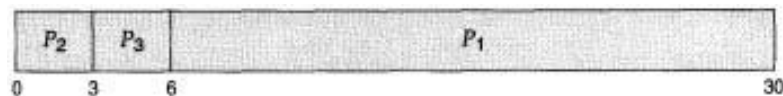
Process	Burst Time
P1	24
P2	3
P3	3

If the processes arrive in the order P1, P2, P3, and are served in FCFS order, we get the result shown in the



following Gantt chart:

The waiting time is 0 milliseconds for process P1, 24 milliseconds for process P2, and 27 milliseconds for process P3. Thus, the average waiting time is $(0 + 24 + 27)/3 = 17$ milliseconds. If the processes arrive in the order P2, P3, P1, however, the results will be as shown in the following Gantt chart:



The average waiting time is now $(6 + 0 + 3)/3 = 3$ milliseconds. This reduction is substantial. Thus, the average waiting time under a FCFS policy is generally not minimal, and may vary substantially if the process CPU-burst times vary greatly.

In addition, consider the performance of FCFS scheduling in a dynamic situation. Assume we have one CPU-bound process and many I/O-bound processes. As the processes flow around the system, the following scenario may result. The CPU-bound process will get the CPU and hold it. During this time, all the other processes will finish their I/O and move into the ready queue, waiting for the CPU. While the processes wait in the ready queue, the I/O devices are idle. Eventually, the CPU-bound process finishes its CPU burst and moves to an I/O device. All the I/O-bound processes, which have very short CPU bursts, execute quickly and move back to the I/O queues. At this point, the CPU sits idle. The CPU-bound process will then move back to the ready queue and be allocated the CPU. Again, all the I/O processes end up waiting in the ready queue until the CPU-bound process is done. There is a convoy effect, as all the other processes wait for the one big process to get off the CPU. This effect results in lower CPU and device utilization than might be possible if the shorter processes were allowed to

go first The FCFS scheduling algorithm is non-preemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O. The FCFS algorithm is particularly troublesome for time-sharing systems, where each user needs to get a share of the CPU at regular intervals. It would be disastrous to allow one process to keep the CPU for an extended period.

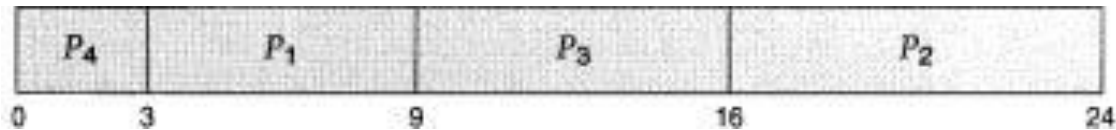
Shortest-Job-First Scheduling

A different approach to CPU scheduling is the shortest-job-first (SJF) scheduling algorithm. This algorithm associates with each process the length of the latter's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If two processes have the same length next CPU burst, FCFS scheduling is used to break the tie. Note that a more appropriate term would be the shortest next CPU burst, because the scheduling is done by examining the length of the next CPU burst of a process, rather than its total length. We use the term SJF because most people and textbooks refer to this type of scheduling discipline as SJF.

As an example, consider the following set of processes, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time
P1	6
p2	8
p3	7
p4	3

Using SJF scheduling, we would schedule these processes according to the following Gantt chart:



The waiting time is 3 milliseconds for process P1, 16 milliseconds for process P2, 9 milliseconds for process P3, and 0 milliseconds for process P4. Thus, the average waiting time is $(3 + 16 + 9 + 0)/4 = 7$ milliseconds. If we were using the FCFS scheduling scheme, then the average waiting time would be 10.25 milliseconds.

The SJF scheduling algorithm is provably optimal, in that it gives the minimum average waiting time for a given set of processes. By moving a short process before a long one, the waiting time of the short process decreases more than it increases the waiting time of the long process. Consequently, the average waiting time decreases.

The real difficulty with the SJF algorithm is knowing the length of the next CPU request. For long-term (or job) scheduling in a batch system, we can use as the length the process time limit that a user specifies when he submits the job.

Thus, users are motivated to estimate the process time limit accurately, since a lower value may mean faster response. (Too low a value will cause a time-limit exceeded error and require resubmission.) SJF scheduling is used frequently in long-term scheduling.

Although the SJF algorithm is optimal, it cannot be implemented at the level of short-term CPU scheduling. There is no way to know the length of the next CPU burst. One approach is to try to approximate SJF scheduling. We may not know the length of the next CPU burst, but we may be able to predict its value. We expect that the next CPU burst will be similar in length to the previous ones.

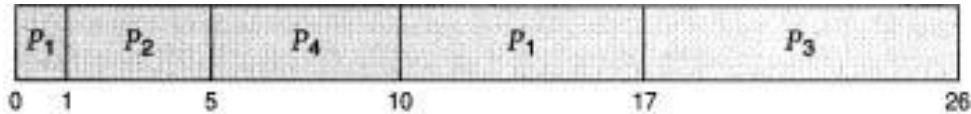
Thus, by computing an approximation of the length of the next CPU burst, we can pick the process with the shortest predicted CPU burst.

The SJF algorithm may be either preemptive or non-preemptive. The choice arises when a new process arrives at the ready queue while a previous process is executing. The new process may have a shorter next CPU burst than what is left of the currently executing process. A preemptive SJF algorithm will preempt the currently executing process, whereas a non-preemptive SJF algorithm will allow the currently running process to finish its CPU burst. Preemptive SJF scheduling is sometimes called shortest-remaining-time-first scheduling.

an example, consider the following four processes, with the length of the CPU-burst time given in milliseconds:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
p4	3	5

If the processes arrive at the ready queue at the times shown and need the indicated burst times, then the resulting preemptive SJF schedule is as depicted in the following Gantt chart:



Process P1 is started at time 0, since it is the only process in the queue. Process P2 arrives at time 1. The remaining time for process P1 (7 milliseconds) is larger than the time required by process P2 (4 milliseconds), so process P1 is preempted, and process P2 is scheduled. The average waiting time for this example is $((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3))/4 = 26/4 = 6.5$ milliseconds. A non-preemptive SJF scheduling would result in an average waiting time of 7.75 milliseconds.

Priority Scheduling

The SJF algorithm is a special case of the general priority-scheduling algorithm. A priority is associated with each process, and the CPU is allocated to the process with the highest priority. Equal-priority processes are scheduled in FCFS order.

An SJF algorithm is simply a priority algorithm where the priority (p) is the inverse of the (predicted) next CPU burst. The larger the CPU burst, the lower the priority, and vice versa.

Note that we discuss scheduling in terms of high priority and low priority. Priorities are generally some fixed ranges of numbers, such as 0 to 7, or 0 to 4,095. However, there is no general agreement on whether 0 is the highest or lowest priority. Some systems use low numbers to represent low priority; others use low numbers for high priority. This difference can lead to confusion. In this text, we use low numbers to represent high priority. As an example, consider the following set of processes, assumed to have arrived at time 0, in the order P1, P2, ..., P5, with the length of the CPU-burst time given in milliseconds:

Process	Burst	Time Priority
P1	10	3
p2	1	1
p3	2	4
P4	1	5
P5	5	2

Using priority scheduling, we would schedule these processes according to the following Gantt chart:



The average waiting time is 8.2 milliseconds. Priorities can be defined either internally or externally. Internally defined priorities use some measurable quantity or quantities to compute the priority of a process.

For example, time limits, memory requirements, the number of open files, and the ratio of average I/O burst to average CPU burst have been used in computing priorities. External priorities are set by criteria that are external to the operating system, such as the importance of the process, the type and amount of funds being paid for computer use, the department sponsoring the work, and other, often political, factors.

Priority scheduling can be either preemptive or non-preemptive. When a process arrives at the ready queue, its priority is compared with the priority of the currently running process. A preemptive priority-scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process. A non-preemptive priority-scheduling algorithm will simply put the new process at the head of the ready queue.

A major problem with priority-scheduling algorithms is indefinite blocking (or starvation). A process that is ready to run but lacking the CPU can be considered blocked-waiting for the CPU. A priority-scheduling algorithm can leave some low-priority processes waiting indefinitely for the CPU.

A solution to the problem of indefinite blockage of low-priority processes is aging. Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time. For example, if priorities range from 127 (low) to 0 (high), we could decrement the priority of a waiting process by 1 every 15 minutes. Eventually, even a process with an initial priority of 127 would have the highest priority in the system and would be executed. In fact, it would take no more than 32 hours for a priority 127 process to age to a priority 0 process.

Round-Robin Scheduling

The round-robin (RR) scheduling algorithm is designed especially for timesharing systems. It is similar to FCFS scheduling, but preemption is added to switch between processes. A small unit of time, called a time quantum (or time slice), is defined. A time quantum is generally from 10 to 100 milliseconds. The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

To implement RR scheduling, we keep the ready queue as a FIFO queue of processes. New processes are added to the tail of the ready queue. The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.

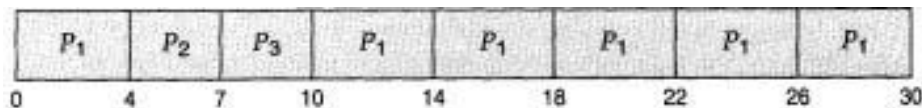
One of two things will then happen. The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue. Otherwise, if the CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the operating system. A context switch will be executed.

The average waiting time under the RR policy, however, is often quite long.

Consider the following set of processes that arrive at time 0, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time
P1	24
P2	3
P3	3

If we use a time quantum of 4 milliseconds, then process P1 gets the first 4 milliseconds. Since it requires another 20 milliseconds, it is preempted after the first-time quantum, and the CPU is given to the next process in the queue, process P2. Since process P2 does not need 4 milliseconds, it quits before its time quantum expires. The CPU is then given to the next process, process P3. Once each process has received 1 time quantum, the CPU is returned to process P1 for an additional time quantum. The resulting RR schedule is



The average waiting time is $17/3 = 5.66$ milliseconds.

In the RR scheduling algorithm, no process is allocated the CPU for more than 1 time quantum in a row. If a process' CPU burst exceeds 1 time quantum, that process is preempted and is put back in the ready queue. The RR scheduling algorithm is preemptive.

If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units. Each process must wait no longer than $(n - 1) \times q$ time units until its next time quantum. For example, if there are five processes, with a time quantum of 20 milliseconds, then each process will get up to 20 milliseconds every 100 milliseconds.

The performance of the RR algorithm depends heavily on the size of the time quantum. At one extreme, if the time quantum is very large (infinite), the RR policy is the same as the FCFS policy. If the time quantum is very small (say 1 microsecond), the RR approach is called processor sharing, and appears (in theory) to the users as though each of n processes has its own processor running at $1/n$ the speed of the real processor. This approach was used in Control Data Corporation (CDC) hardware to implement 10 peripheral processors with only one set of hardware and 10 sets of registers. The hardware executes one instruction for one set of registers, then goes on to the next. This cycle continues, resulting in 10 slow processors rather than one fast processor.

Multilevel Queue Scheduling

Another class of scheduling algorithms has been created for situations in which processes are easily classified into different groups. For example, a common division is made between foreground (or interactive) processes and background (or batch) processes. These two types of processes have different response-time requirements, and so might have different scheduling needs. In addition, foreground processes may have priority (or externally defined) over background processes.

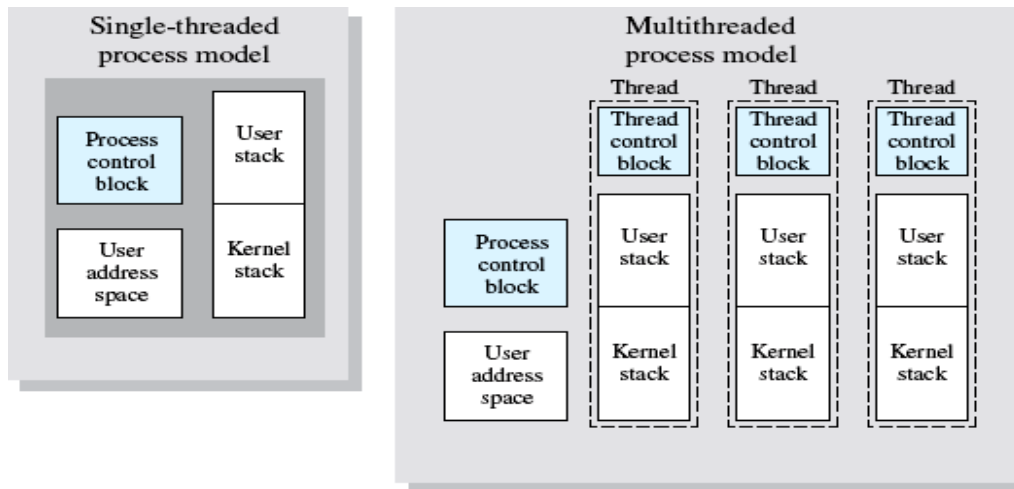
Thread

A thread, sometimes called a lightweight process (LWP), is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals. A traditional (or heavyweight) process has a single thread of control. If the process has multiple threads of control, it can do more than one task at a time.

Motivation

Many software packages that run on modern desktop PCs are multithreaded. An application typically is implemented as a separate process with several thread of control.

Single-threaded and multithreaded



Ex: A web browser might have one thread display images or text while another thread retrieves data from the network. A word processor may have a thread for displaying graphics, another thread for reading keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

In certain situations, a single application may be required to perform several similar tasks. For example, a web server accepts client requests for web pages, images, sound, and so forth. A busy web server may have several (perhaps hundreds) of clients concurrently accessing it. If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time.

One solution is to have the server run as a single process that accepts requests. When the server receives a request, it creates a separate process to service that request. In fact, this process-creation method was in common use before threads became popular. Process creation is very heavyweight, as was shown in the previous chapter. If the new process will perform the same tasks as the existing process, why incur all that overhead? It is generally more efficient for one process that contains multiple threads to serve the same purpose. This approach would multithread the web-server process. The server would create a separate thread that would listen for client requests; when a request was made, rather than creating another process, it would create another thread to service the request.

Threads also play a vital role in remote procedure call (RPC) systems. RPCs allow inter-process communication by providing a communication mechanism similar to ordinary function or procedure calls. Typically, RPC servers are multithreaded. When a server receives a message, it services the message using a separate thread. This allows the server to service several concurrent requests.

Benefits

The benefits of multithreaded programming can be broken down into four major categories:

1. Responsiveness: Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. For instance, a multithreaded web browser could still allow user interaction in one thread while an image is being loaded in another thread.

2. Resource sharing: By default, threads share the memory and the resources of the process to which they belong. The benefit of code sharing is that it allows an application to have several different threads of activity all within the same address space.

3. Economy: Allocating memory and resources for process creation is costly. Alternatively, because threads share resources of the process to which they belong, it is more economical to create and context switch threads. It can be difficult to gauge empirically the difference in overhead for creating and maintaining a process rather than a thread, but in general it is much more time consuming to create and manage processes than threads. In Solaris 2, creating a process is about 30 times slower than is creating a thread, and context switching is about five times slower.

4. Utilization of multiprocessor architectures: The benefits of multithreading can be greatly increased in a multiprocessor architecture, where each thread may be running in parallel on a different processor. A single-threaded process can only run on one CPU, no matter how many are available.

Multithreading on a multi-CPU machine increases concurrency. In a single processor architecture, the CPU generally moves between each thread so quickly as to create an illusion of parallelism, but in reality, only one thread is running at a time.

The OS supports the threads that can be provided in following two levels:

User-Level Threads

User-level threads implement in user-level libraries, rather than via systems calls, so thread switching does not need to call operating system and to cause interrupt to the kernel. In fact, the kernel knows nothing about user-level threads and manages them as if they were single-threaded processes.

Advantages:

- User-level threads do not require modification to operating systems.
- Simple Representation: Each thread is represented simply by a PC, registers, stack and a small control block, all stored in the user process address space.
- Simple Management: This simply means that creating a thread, switching between threads and synchronization between threads can all be done without intervention of the kernel.
- Fast and Efficient: Thread switching is not much more expensive than a procedure call.

Disadvantages:

- There is a lack of coordination between threads and operating system kernel.
- User-level threads require non-blocking systems call i.e., a multithreaded kernel.

Kernel-Level Threads

In this method, the kernel knows about and manages the threads. Instead of thread table in each process, the kernel has a thread table that keeps track of all threads in the system. Operating Systems kernel provides system call to create and manage threads.

Advantages:

- Because kernel has full knowledge of all threads, Scheduler may decide to give more time to a process having large number of threads than process having small number of threads.
- Kernel-level threads are especially good for applications that frequently block.

Disadvantages:

- The kernel-level threads are slow and inefficient. For instance, threads operations are hundreds of times slower than that of user-level threads.

Since kernel must manage and schedule threads as well as processes. It requires a full thread control block (TCB) for each thread to maintain information about threads. As a result, there is significant overhead and increased in kernel complexity.

Multithreading Models

Many systems provide support for both user and kernel threads, resulting in different multithreading models. We look at three common types of threading implementation.

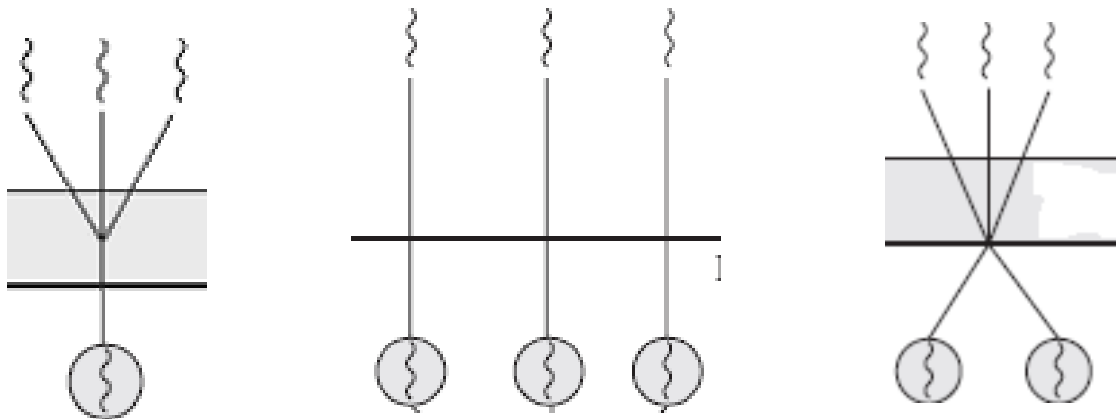
Many-to-One Model

The many-to-one model maps many user-level threads to one kernel thread. Thread management is done in user space, so it is efficient, but the entire process will block if a thread makes a blocking system call. Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors. One-to-one Model

The one-to-one model maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call; it also allows multiple threads to run in parallel on multiprocessors. The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread. Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads supported by the system. Windows NT, Windows 2000, and OS/2 implement the one-to-one model.

Many-to-Many Model

The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads. The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a multiprocessor than on a uniprocessor). Whereas the many-to-one model allows the developer to create as many user threads as she wishes, true concurrency is not gained because the kernel can schedule only one thread at a time. The one-to-one model allows for greater concurrency, but the developer has to be careful not to create too many threads within an application (and in some instances may be limited in the number of threads she can create).



(Diagram of many-to-one model, one-to-one model and many-to-many model)

Process vs. Thread

Process	Thread
1.Process cannot share the same memory area (address space)	1.Threads can share memory and files.
2.It takes more time to create a process	2.It takes less time to create a thread.
3.It takes more time to complete the execution and terminate.	3.Less time to terminate.
4.Execution is very slow.	4.Execution is very fast.
5.It takes more time to switch between two processes.	5.It takes less time to switch between two threads.
6.System calls are required to communicate each other	6.System calls are not required.
7.It requires more resources to execute.	7.Requires fewer resources.
8.Implementing the communication between processes is bit more difficult.	8.Communication between two threads are very easy to implement because threads share the memory



Pointer-process Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions.
- Message system – processes communicate with each other without resorting to shared variables.
- IPC facility provides two operations:
 1. **send**(*message*) – message size fixed or variable
 2. **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
 1. establish a *communication link* between them
 2. exchange messages via send/receive
- Implementation of communication link
 1. physical (e.g., shared memory, hardware bus)
 2. logical (e.g., logical properties)

Direct Communication

Processes must name each other explicitly:

- ◆ **send** (*P*, *message*) – send a message to process *P*
- ◆ **receive** (*Q*, *message*) – receive a message from process *Q* Properties of communication link
- ◆ Links are established automatically.
- ◆ A link is associated with exactly one pair of communicating processes.
- ◆ Between each pair there exists exactly one link.
- ◆ The link may be unidirectional, but is usually bi-directional.

Indirect Communication

Messages are directed and received from mailboxes (also referred to as ports).

- ◆ Each mailbox has a unique id.
- ◆ Processes can communicate only if they share a mailbox.

Properties of communication link

- ◆ Link established only if processes share a common mailbox
- ◆ A link may be associated with many processes.
- ◆ Each pair of processes may share several communication links.
- ◆ Link may be unidirectional or bi-directional.

Operations

- ◆ create a new mailbox
- ◆ send and receive messages through mailbox
- ◆ destroy a mailbox

Primitives are defined as:

send (*A*, *message*) – send a message to mailbox *A*

receive (*A*, *message*) – receive a message from mailbox *A*

Mailbox sharing

- ◆ *P1*, *P2*, and *P3* share mailbox *A*.
- ◆ *P1*, sends; *P2* and *P3* receive.
- ◆ Who gets the message?

Solutions

- ◆ Allow a link to be associated with at most two processes.
- ◆ Allow only one process at a time to execute a receive operation.
- ◆ Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Concurrent Processes

The concurrent processes executing in the operating system may be either independent processes or cooperating processes. A process is independent if it cannot affect or be affected by the other processes executing in the system. Clearly, any process that does not share any data (temporary or persistent) with any other process is independent. On the other hand, a process is cooperating if it can affect or be affected by the other processes

executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

We may want to provide an environment that allows process cooperation for several reasons:

Information sharing: Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to these types of resources.

Computation speedup: If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Such a speedup can be achieved only if the computer has multiple processing elements (such as CPUs or I/O channels).

Modularity: We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.

Convenience: Even an individual user may have many tasks on which to work at one time. For instance, a user may be editing, printing, and compiling in parallel.

Message-Passing System

The function of a message system is to allow processes to communicate with one another without the need to resort to shared data. We have already seen message passing used as a method of communication in microkernels. In this scheme, services are provided as ordinary user processes. That is, the services operate outside of the kernel. Communication among the user processes is accomplished through the passing of messages. An IPC facility provides at least the two operations: send (message) and receive (message).

Messages sent by a process can be of either fixed or variable size. If only fixed-sized messages can be sent, the system-level implementation is straightforward. This restriction, however, makes the task of programming more difficult. On the other hand, variable-sized messages require a more complex system-level implementation, but the programming task becomes simpler.

If processes P and Q want to communicate, they must send messages to and receive messages from each other; a communication link must exist between them. This link can be implemented in a variety of ways. We are concerned here not with the link's physical implementation, but rather with its logical implementation. Here are several methods for logically implementing a link and the send/receive operations:

- Direct or indirect communication
- Symmetric or asymmetric communication
- Automatic or explicit buffering
- Send by copy or send by reference
- Fixed-sized or variable-sized messages

We look at each of these types of message systems next.

Synchronization

Communication between processes takes place by calls to send and receive primitives. There are different design options for implementing each primitive. Message passing may be either blocking or nonblocking—also known as synchronous and asynchronous.

Blocking send: The sending process is blocked until the message is received by the receiving process or by the mailbox.

Nonblocking send: The sending process sends the message and resumes operation.

Blocking receives: The receiver blocks until a message is available.

Nonblocking receive: The receiver retrieves either a valid message or a null.

Different combinations of send and receive are possible. When both the send and receive are blocking, we have a rendezvous between the sender and the receiver.

Buffering

Whether the communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue. Basically, such a queue can be implemented in three ways:

Zero capacity: The queue has maximum length 0; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.

Bounded capacity: The queue has finite length n ; thus, at most n messages can reside in it. If the queue is not full when a new message is sent, the latter is placed in the queue (either the message is copied or a pointer to the message is kept), and the sender can continue execution without waiting. The link has a finite capacity, however. If the link is full, the sender must block until space is available in the queue.

Unbounded capacity: The queue has potentially infinite length; thus, any number of messages can wait in it. The sender never blocks. The zero-capacity case is sometimes referred to as a message system with no buffering; the other cases are referred to as automatic buffering.

Client-Server Communication

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

Process Synchronization

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a race condition.

Producer-Consumer Problem

Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process.

To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced. In this situation, the consumer must wait until an item is produced.

- ♦ *unbounded-buffer* places no practical limit on the size of the buffer.
- ♦ *bounded-buffer* assumes that there is a fixed buffer size.

Bounded-Buffer – Shared-Memory Solution

The consumer and producer processes share the following variables.

Shared data

```
#define BUFFER_SIZE 10
typedef struct
{
    ...
}
item;
item buffer
[BUFFER_SIZE]; int in = 0;
int out = 0;
```

Solution is correct, but can only use BUFFER_SIZE-1 elements.

The shared buffer is implemented as a circular array with two logical pointers: in and out. The variable in points to the next free position in the buffer; out points to the first full position in the buffer. The buffer is empty when in == out; the buffer is full when $((in + 1) \% BUFFER_SIZE) == out$.

The code for the producer and consumer processes follows. The producer process has a local variable **nextProduced** in which the new item to be produced is stored:

Bounded-Buffer – Producer Process

```
item nextProduced;
while (1)
{
    while (((in + 1) \% BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) \% BUFFER_SIZE;
}
```

Bounded-Buffer – Consumer Process

```
item nextConsumed;

while (1)
{
    While (in == out);

    /* do nothing */ nextConsumed = buffer[out];

    Out = (out + 1) % BUFFER_SIZE;}
```

The critical section problem

Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, writing a file, and so on. The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section. Thus, the execution of critical sections by the processes is **mutually exclusive** in time. The critical-section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section.

```
Do
{
    Entry section
        Critical section
    Exit section
        Remainder section
} while (1);
```

A solution to the critical-section problem must satisfy the following three requirements:

- 1. Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- 2. Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder section can participate in the decision on which will enter its critical section next, and this selection cannot be postponed indefinitely.
- 3. Bounded Waiting:** There exists a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's solution

Peterson's solution is a software-based solution to the critical section problem.

Consider two processes P_0 and P_1 . For convenience, when presenting P_i , we use P_i to denote the other process; that is, $j = 1 - i$.

The processes share two variables: Boolean flag [2];

Int turn;

Initially flag [0] = flag [1] = false, and the value of turn is immaterial (but is either 0 or 1). The structure of process P_i is shown below.

```
Do {
    Flag[i]=true turn=j
```

```

While(flag[j] && turn==j);
Critical section flag[i]=false
Remainder section
} while (1);

```

To enter the critical section, process P_i first sets $\text{flag}[i]$ to be true and then sets turn to the value j , thereby asserting that if the other process wishes to enter the critical section it can do so. If both processes try to enter at the same time, turn will be set to both i and j at roughly the same time. Only one of these assignments will last; the other will occur, but will be overwritten immediately. The eventual value of turn decides which of the two processes is allowed to enter its critical section first.

We now prove that this solution is correct. We need to show that:

1. Mutual exclusion is preserved,
2. The progress requirement is satisfied,
3. The bounded-waiting requirement is met.

To prove property 1, we note that each P_i enters its critical section only if either $\text{flag}[j] == \text{false}$ or $\text{turn} == j$. Also note that, if both processes can be executing in their critical sections at the same time, then $\text{flag}[i] == \text{flag}[j] == \text{true}$. These two observations imply that P_0 and P_1 could not have successfully executed their while statements at about the same time, since the value of turn can be either 0 or 1, but cannot be both. Hence, one of the processes say P_j must have successfully executed the while statement, whereas P_i had to execute at least one additional statement (" $\text{turn} == j$ "). However, since, at that time, $\text{flag}[j] == \text{true}$, and $\text{turn} == j$, and this condition will persist as long as P_i is in its critical section, the result follows:

To prove properties 2 and 3, we note that a process P_i can be prevented from entering the critical section only if it is stuck in the while loop with the condition $\text{flag}[j] == \text{true}$ and $\text{turn} == j$; this loop is the only one. If P_i is not ready to enter the critical section, then $\text{flag}[j] == \text{false}$ and P_i can enter its critical section. If P_i has set $\text{flag}[j]$ to true and is also executing in its while statement, then either $\text{turn} == i$ or $\text{turn} == j$. If $\text{turn} == i$, then P_i will enter the critical section. If $\text{turn} == j$, then P_i will enter the critical section. However, once P_i exits its critical section, it will reset $\text{flag}[j]$ to false, allowing P_i to enter its critical section. If P_i resets $\text{flag}[j]$ to true, it must also set turn to i .

Thus, since P_i does not change the value of the variable turn while executing the while statement, P_i will enter the critical section (progress) after at most one entry by P_j (bounded waiting).

Synchronization Hardware

As with other aspects of software, hardware features can make the programming task easier and improve system efficiency. In this section, we present some simple hardware instructions that are available on many systems, and show how they can be used effectively in solving the critical-section problem.

The definition of the TestAndSet instruction.

```

Boolean TestAndSet(boolean &target)
{
    Boolean rv = target; target = true;
    Return rv;
}

```

The critical-section problem could be solved simply in a uniprocessor environment if we could forbid Interrupts to occur while a shared variable is being modified. In this manner, we could be sure that the current sequence of instructions would be allowed to execute in order without preemption. No other instructions would be run, so no unexpected modifications could be made to the shared variable.

Unfortunately, this solution is not feasible in a multiprocessor environment. Disabling interrupts on a multiprocessor can be time-consuming, as the message is passed to all the processors. This message passing delays entry into each critical section, and system efficiency decreases. Also, consider the effect on a system's clock, if the clock is kept updated by interrupts.

Many machines therefore provide special hardware instructions that allow us either to test and modify the content of a word, or to swap the contents of two words, atomically—that is, as one uninterruptible unit. We can use these special instructions to solve the critical-section problem in a relatively simple manner. Rather than discussing one specific

instruction for one specific machine, let us abstract the main concepts behind these types of instructions.

The TestAndSet instruction can be defined as shown in code. The important characteristic is that this instruction is executed atomically. Thus, if two TestAndSet instructions are executed simultaneously (each on a different CPU), they will be executed sequentially in some arbitrary order.

Mutual-exclusion implementation with TestAndSet

```
Do {
While (TestAndSet(lock));
Critical section lock=false
Remainder section
}
While (1);
Void Swap (boolean &a, boolean &b) {boolean temp = a;
A = b;
B = temp}
```

If the machine supports the TestAndSet instruction, then we can implement mutual exclusion by declaring a Boolean variable lock, initialized too false.

If the machine supports the Swap instruction, then mutual exclusion can be provided as follows. A global Boolean variable lock is declared and is initialized too false. In addition, each process also has a local Boolean variable key.

Semaphores

The solutions to the critical-section problem presented before are not easy to generalize to more complex problems. To overcome this difficulty, we can use a synchronization tool called a semaphore. A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait and signal. These operations were originally termed P (for wait; from the Dutch proberen, to test) and V (for signal; from verhogen, to increment). The classical definition of wait in pseudocode is

```
Wait(S) {while (S <= 0)
; // no-op S --;
}
```

The classical definitions of signal in pseudocode are

```
Signal(S) {S++;
}
```

Modifications to the integer value of the semaphore in the wait and signal operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value. In addition, in the case of the wait (S), the testing of the integer value of S ($S \leq 0$), and its possible modification ($S--$), must also be executed without interruption.

Usage

We can use semaphores to deal with the n-process critical-section problem. The n processes share a semaphore, mutex (standing for mutual exclusion), initialized to 1. Each process P_i is organized as shown in Figure. We can also use semaphores to solve various synchronization problems.

For example, consider two concurrently running processes: P_1 with a statement S_1 and P_2 with a statement S_2 . Suppose that we require that S_2 be executed only after S_1 has completed. We can implement this scheme readily by letting P_1 and P_2 share a common semaphore synch, initialized to 0, and by inserting the statements in process P_1 , and the statements

```
Wait (synch); s2;
In process P2. Because synch is initialized to 0, P2 will execute S2 only after P1 has invoked signal (synch), which is after S1.
```

```
S1;
```

```

Signal (synch); do {
Wait (mutex);
Critical section signal (mutex);
Remainder section
} while (1);

```

Mutual-exclusion implementation with semaphores.

Implementation

The main disadvantage of the mutual-exclusion solutions and of the semaphore definition given here, is that they all require busy waiting. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes. Busy waiting wastes CPU cycles that some other process might be able to use productively. This type of semaphore is also called a spinlock (because the process "spins" while waiting for the lock). Spinlocks are useful in multiprocessor systems. The advantage of a spinlock is that no context switch is required when a process must wait on a lock, and a context switch may take considerable time. Thus, when locks are expected to be held for short times, spinlocks are useful. `Int C;`

Initially $S_1 = 1$, $S_2 = 0$, and the value of integer C is set to the initial value of the counting semaphore S . The wait operation on the counting semaphore S can be implemented as follows:

```

Wait (S1);
C--;
If (C < 0) {
Signal(S1); wait (S2);
}
Signal(S1);

```

The signal operation on the counting semaphore S can be implemented as follows: `wait (S1);`

```

C++;
If (C <= 0)
Signal (S2); e l s e signal (S1);

```

Classic Problems of Synchronization

We present a number of different synchronization problems as examples for a large class of concurrency-control problems. These problems are used for testing nearly every newly proposed synchronization scheme. Semaphores are used for synchronization in our solutions.

The Bounded-Buffer Problem

The bounded-buffer problem is commonly used to illustrate the power of synchronization primitives. We present here a general structure of this scheme, without committing ourselves to any particular implementation. We assume that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The empty and full semaphores count the number of empty and full buffers, respectively. The semaphore empty is initialized to the value n ; the semaphore full is initialized to the value 0.

The code for the producer process is

```

Do {
Produce an item in nextp
...
Wait (empty); wait (mutex);
...
Add nextp to buffer
...
Signal(mutex); signal (full);
} While (1);
The code for the consumer process is do {
Wait (full); wait (mutex);

```



```

...
remove an item from buffer to nextc
...
signal (mutex); signal (empty);
...
consume the item in nextc
...
) while (1);

```

Note the symmetry between the producer and the consumer. We can interpret this code as the producer producing full buffers for the consumer, or as the consumer producing empty buffers for the producer.

The Readers- Writers Problem

A data object (such as a file or record) is to be shared among several concurrent processes. Some of these processes may want only to read the content of the shared object, whereas others may want to update (that is, to read and write) the shared object. We distinguish between these two types of processes by referring to those processes that are interested in only reading as readers, and to the rest as writers. Obviously, if two readers access the shared data object simultaneously, no adverse effects will result. However, if a writer and some other process (either a reader or a writer) access the shared object simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared object. This synchronization problem is referred to as the readers-writers problem. Since it was originally stated, it has been used to test nearly every new synchronization primitive. The readers-writers problem has several variations, all involving priorities. The simplest one, referred to as the first readers-writers problem, requires that no reader will be kept waiting unless a writer has already obtained permission to use the shared object. In other words, no reader should wait for other readers to finish simply because a writer is waiting. The second readers-writers problem requires that, once a writer is ready, that writer performs its write as soon as possible. In other words, if a writer is waiting to access the object, no new readers may start reading.

A solution to either problem may result in starvation. In the first case, writers may starve; in the second case, readers may starve. For this reason, other variants of the problem have been proposed. In this section, we present a solution to the first readers-writers problem. In the solution to the first readers-writers problem, the reader processes share the following data structures:

```

semaphore mutex, wrt;
int readcount;

```

The semaphores `mutex` and `wrt` are initialized to 1; `readcount` is initialized to 0. The semaphore `wrt` is common to both the reader and writer processes. The `mutex` semaphore is used to ensure mutual exclusion when the variable `readcount` is updated. The `readcount` variable keeps track of how many processes are currently reading the object. The semaphore `wrt` functions as a mutual-exclusion semaphore for the writers. It is also used by the first or last reader that enters or exits the critical section. It is not used by readers who enter or exit while other readers are in their critical sections.

The code for a writer process is

```

Do {
wait (wrt);
...
writing is performed
...
signal(wrt);
} while (1);

```

The code for a reader process is

```

Do {
wait (mutex);
readcount++;
if (readcount == 1)
wait (wrt);
signal (mutex);
...

```

reading is performed

```
...
wait (mutex);
readcount--;
if (readcount == 0)
signal (wrt1;
signal (mutex);
} while (1);
```

The Dining-Philosophers Problem

Consider five philosophers who spend their lives thinking and eating. The philosophers share a common circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chopsticks. When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors). A philosopher may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick that is already in the hand of a neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.

The structure of philosopher i

```
do {
wait (chopstick[i]);
wait (chopstick[(i+1) % 5]);
...
eat
...
signal (chopstick [i];
signal(chopstick[(i+1) % 5]);
...
think
...
} while (1);
```

The dining-philosophers problem is considered a classic synchronization problem, neither because of its practical importance nor because computer scientists dislike philosophers, but because it is an example of a large class of concurrency-control problems. It is a simple representation of the need to allocate several resources among several processes in a deadlock- and starvation free manner.

One simple solution is to represent each chopstick by a semaphore. A philosopher tries to grab the chopstick by executing a wait operation on that semaphore; she releases her chopsticks by executing the signal operation on the appropriate semaphores. Thus, the shared data are semaphore chopstick [5]; where all the elements of chopstick are initialized to 1.

Although this solution guarantees that no two neighbors are eating simultaneously,

It nevertheless must be rejected because it has the possibility of creating a deadlock. Suppose that all five philosophers become hungry simultaneously, and each grabs her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.

Use an asymmetric solution; that is, an odd philosopher picks up first her left chopstick and then her right chopstick, whereas an even philosopher picks up her right chopstick and then her left chopstick. Finally, any satisfactory solution to the dining-philosophers problem must guard against the possibility that one of the philosophers will starve to death.

Monitors

Another high-level synchronization construct is the monitor type. A monitor is characterized by a set of programmer-defined operators. The representation of a monitor type consists of declarations of variables whose values define the state of an instance of the type, as well as the bodies of procedures or functions that implement operations on the type. The syntax of a monitor is.

The representation of a monitor type cannot be used directly by the various processes. Thus, a procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters. Similarly, the local variables of a monitor can be accessed by only the local procedures.

The monitor construct ensures that only one process at a time can be active within the monitor. Consequently, the programmer does not need to code this synchronization constraint explicitly. However, the monitor construct, as defined so far, is not sufficiently powerful for modeling some synchronization schemes. For this purpose, we need to define additional synchronization mechanisms. These mechanisms are provided by the condition operations construct.

A programmer who needs to write her own tailor-made synchronization scheme can define one or more variables of type condition:

condition x, y; The only operations that can be invoked on a condition variable are wait and signal. The operation means that the process invoking this operation is suspended until another process invokes the x signal (1 operation resumes exactly one suspended process. If no process is suspended, then the signal operation has no effect; that is, the state of x is as though the operation were never executed Contrast this operation with the signal operation associated with semaphores, which always affects the state of the semaphore. Now suppose that, when the x. signal () operation is invoked by a process P, there is a suspended process Q associated with condition x. Clearly, if the operations suspended process Q is allowed to resume its execution, the signaling process P must wait. Otherwise, both P and Q will be active simultaneously within the monitor. Note, however, that both processes can conceptually continue with their execution.

Two possibilities exist:

1. P either waits until Q leaves the monitor, or waits for another condition.
2. Q either waits until P leaves the monitor, or waits for another condition.

There are reasonable arguments in favor of adopting either option 1 or option 2. Since P was already executing in the monitor, choice 2 seems more reasonable. However, if we allow process P to continue, the "logical" condition for which Q was waiting may no longer hold by the time Q is resumed. Choice 1 was advocated by Hoare, mainly because the preceding argument in favor of it translates directly to simpler and more elegant proof rules. A compromise between these two choices was adopted in the language Concurrent C. When process P executes the signal operation, process Q is immediately resumed. This model is less powerful than Hoare's, because a process cannot signal more than once during a single procedure call.

